

IMAGS

Music-Induced Analgesia Genome Study

Sean Smith, Charlie Lantz,
Dylan Serreyn, Jadon Lemkin



Credit: Verywell / Lara Antal

IMAGS: Music-Induced Analgesia Genome Study

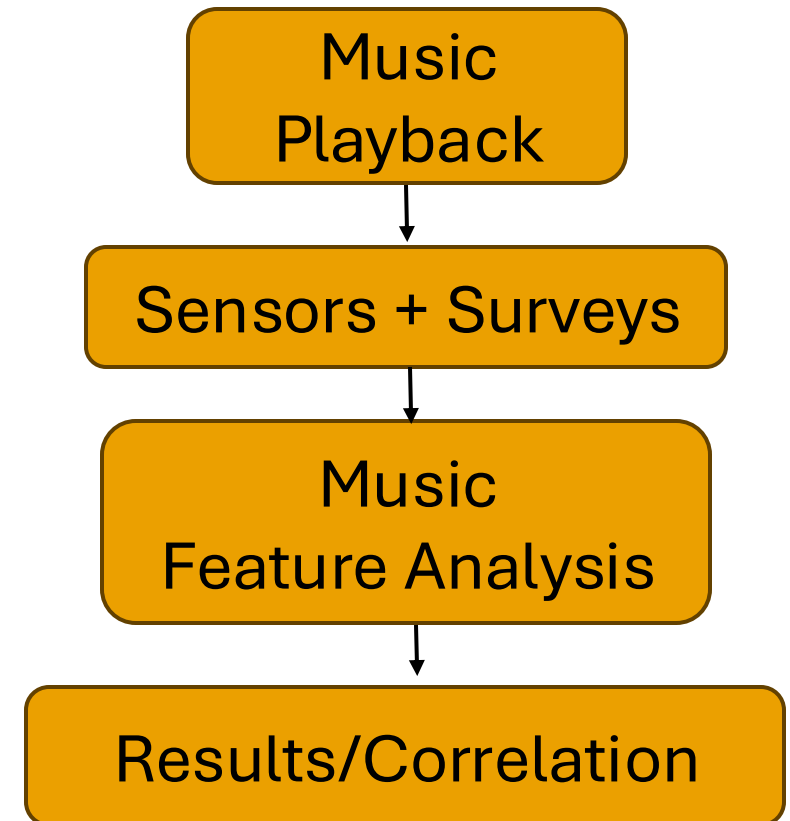
- ♪ How does music reduce chronic pain?
- ♪ Combines music listening, physiological data, and self-reported pain surveys



More Than Just Entertainment by M. Murray, MM, MT-BC, June 29, 2022, NAMI

Project Goals: Expanding IMAGS

- ♪ Analyze musical features at moments of reduced pain awareness
- ♪ Explore different pain indicators/sensors
- ♪ Develop a music interface that shows:
 - ♪ Physiological Responses
 - ♪ Pain Surveys
 - ♪ Extracted Musical Features

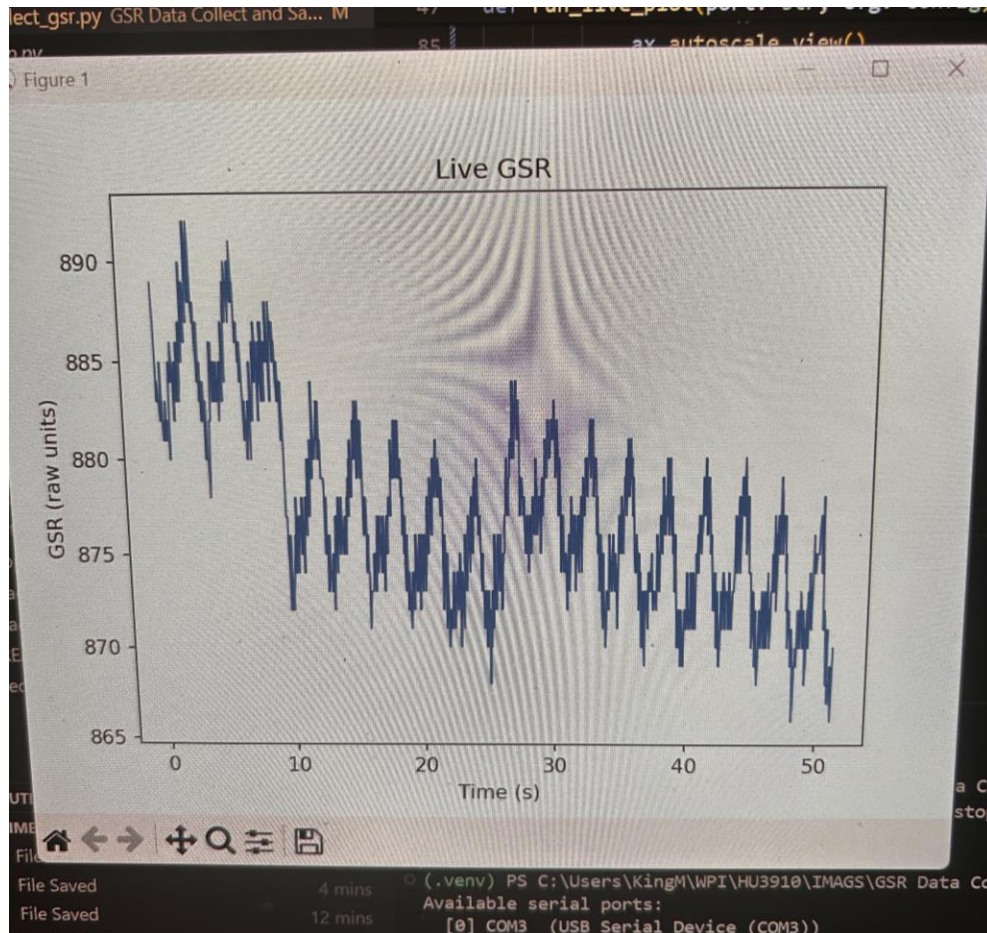




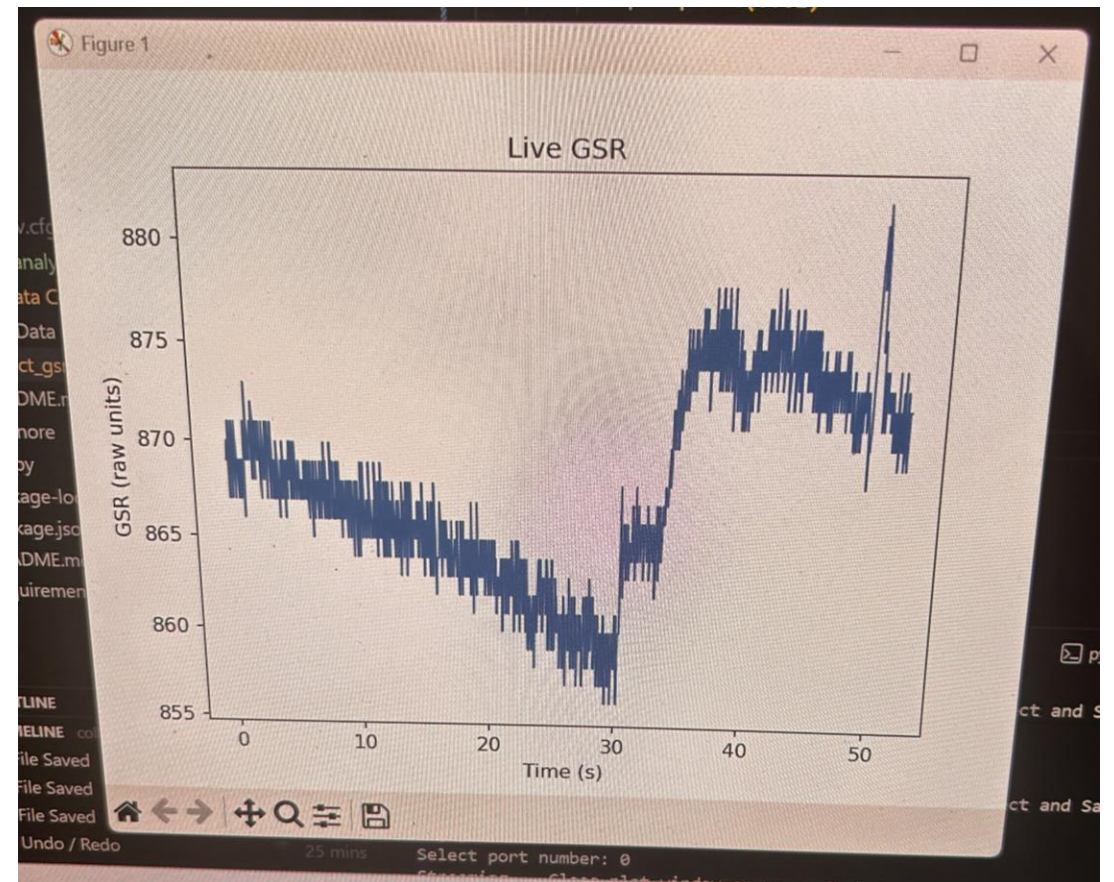
Problems and Adapting

- Starting from scratch
 - New GSR
 - New Software
 - No prior research
- Environmental noise in GSR data

Environmental Noise



Without Charger



With Charger

Member Concentrations

Dylan and
Jadon

Software

Charlie

Research and
Presentations

Sean

Hardware

Music Information Retrieval (MIR)

- Used to determine song characteristics
- Can determine items such as:
 - Key
 - Tempo
 - Volume
 - Mood/Genre
- Our open-source software
 - Essentia

Parts of Music to Measure

- Familiarity
- Global Tempo/BPM
- Mood
- Dynamic Range
- Loudness
- Energy



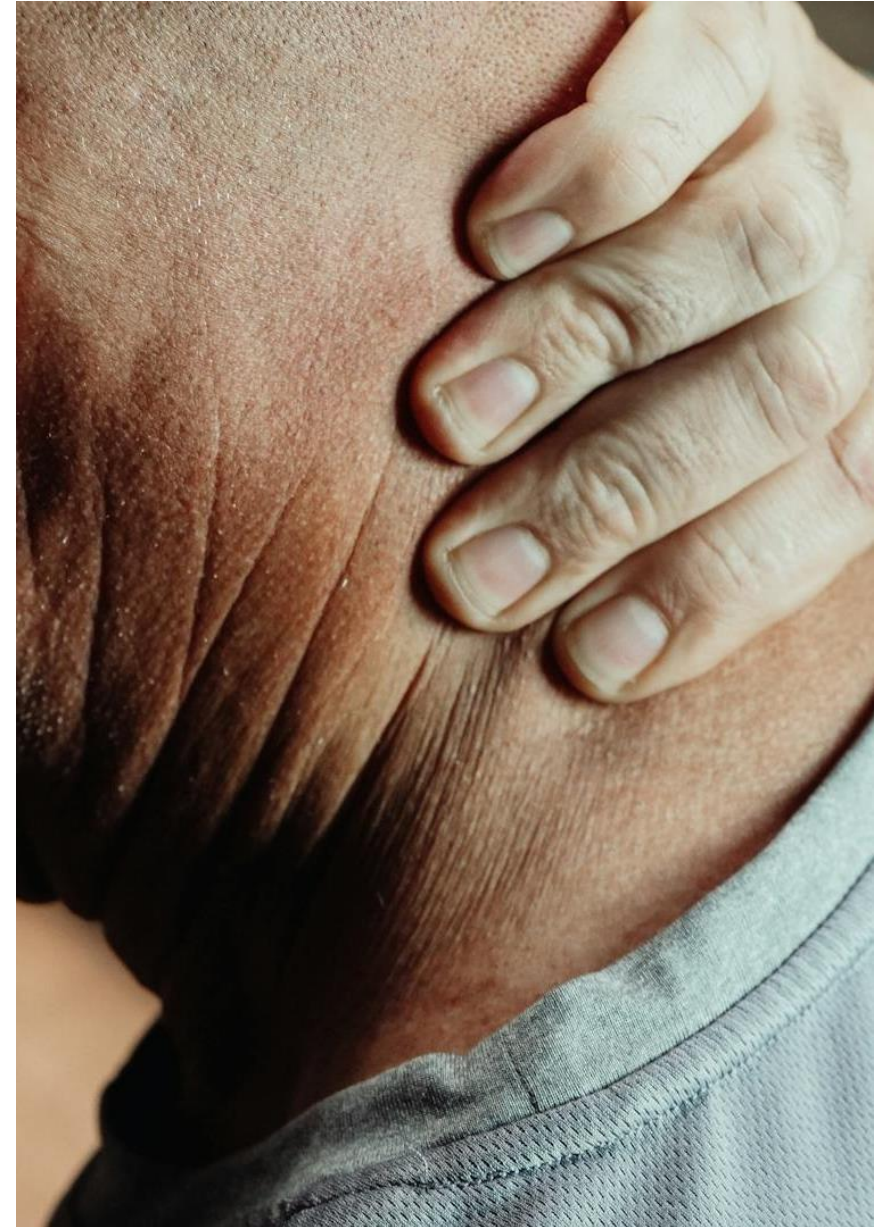
Survey Questions

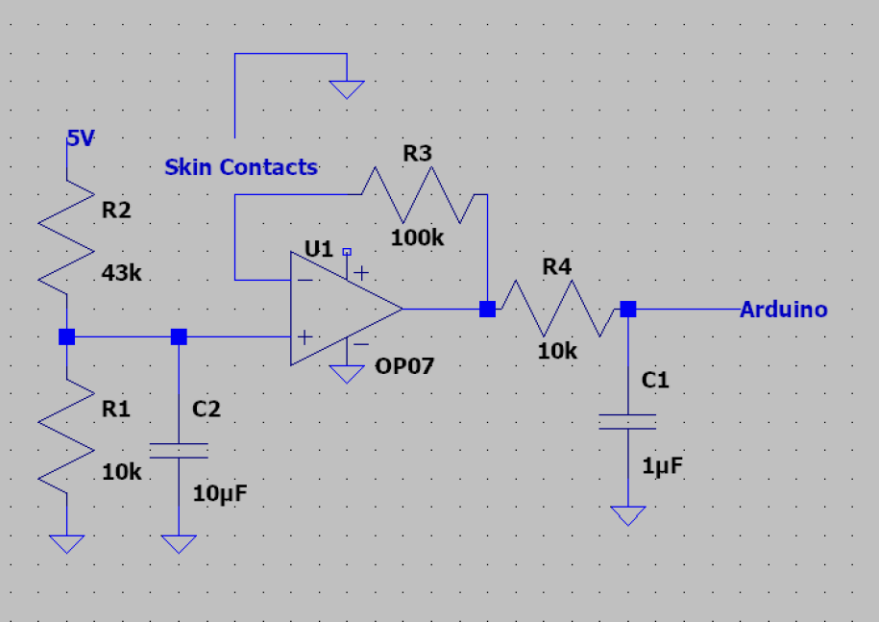
- Prior to testing:
 - Rating pain levels
 - Familiarity and experience with music
- After testing:
 - Rating pain levels and changes
 - Explaining experiences and engagement
- GSR:
 - Environment related such as room temp or time of day
 - Patient related such as last meal or caffeine use



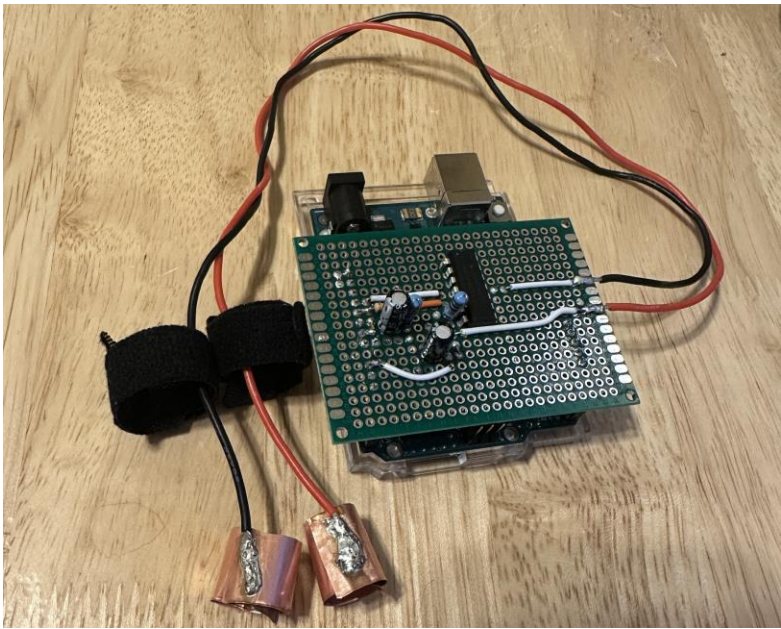
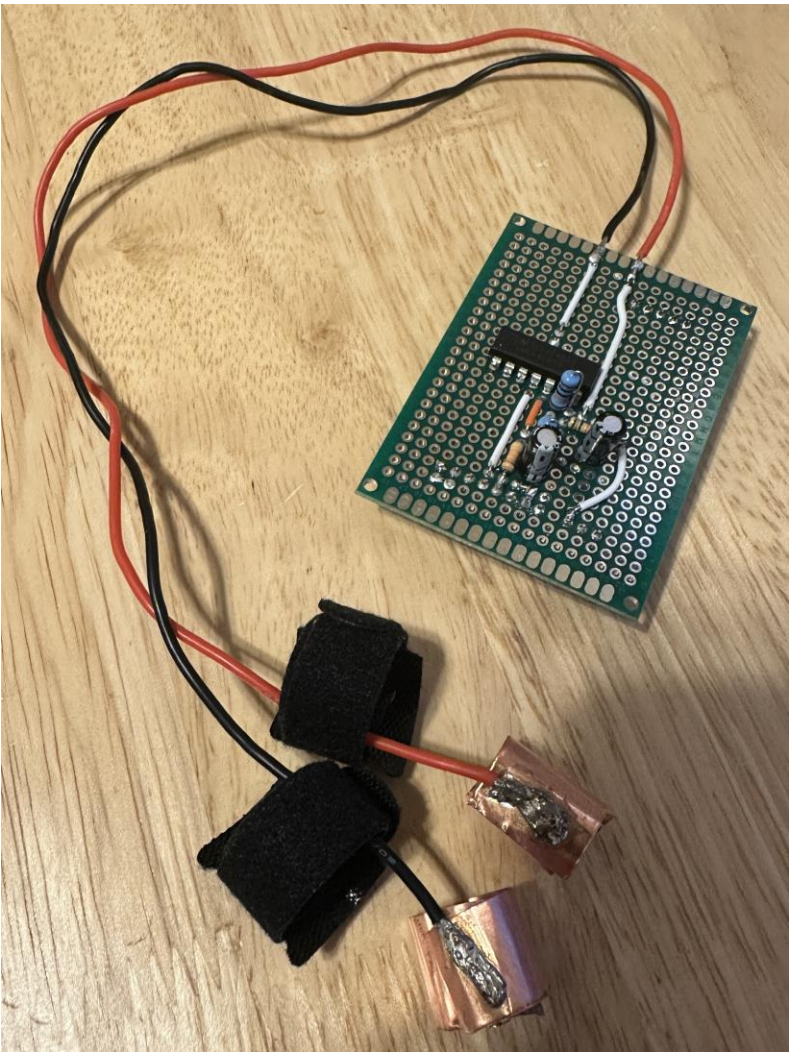
Measuring Chronic Pain

- Measuring pain is difficult, especially in chronic pain patients.
- Chronic pain is intertwined with stress and can sometimes be exacerbated by stress responses.¹
- Both self-reported pain/stress data and physiological indicators are often used to evaluate pain.¹
- To get an accurate measure of perceived pain, we should collate our sensor and user-reported data prior to processing.





Our GSR



Live GSR plotting:

Raw GSR



```
ion View Go Run ... ← → Q IMAGS
y app.py collect_gsr.py U X
ct_gsr.py > run_live_plot
parse_line(line: str) -> Optional[float]:
    return None
if s.lower() in ("start", "stop"):
    return None
try:
    return float(s)
except ValueError:
    return None

run_live_plot(port: str, cfg: Config):
    window_size = 1
    buf = deque(maxlen=window_size)

    ser = serial.Serial(port, cfg.baud, timeout=1)
    time.sleep(2.0) # allow Arduino reset
    ser.reset_input_buffer()

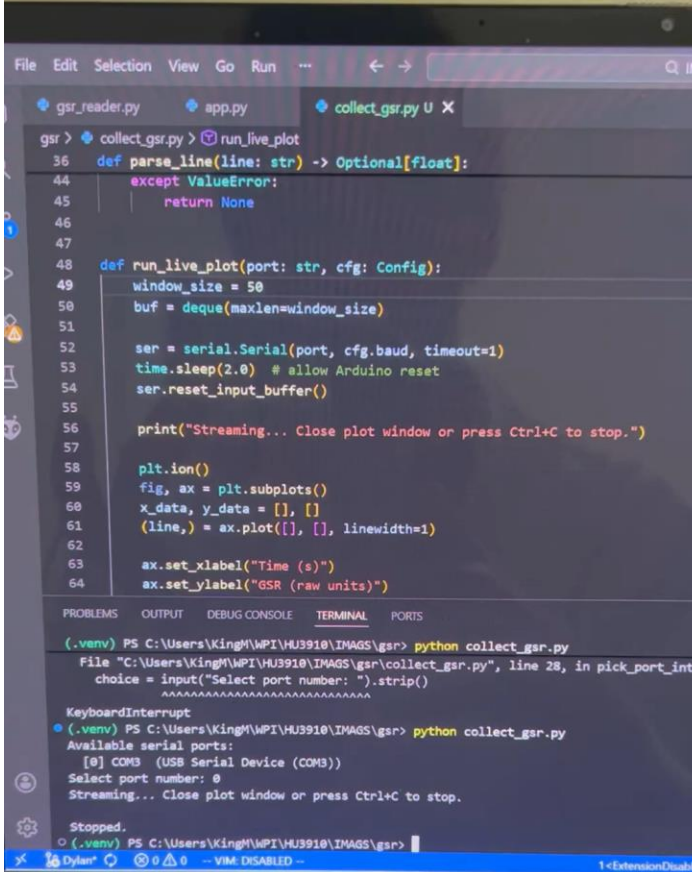
    print("Streaming... Close plot window or press Ctrl+C to stop.")

    plt.ion()
    fig, ax = plt.subplots()

OUTPUT DEBUG CONSOLE TERMINAL PORTS
PS C:\Users\KingM\WPI\HU3910\IMAGS\gsr> python collect_gsr.py
M3 (USB Serial Device (COM3))
port number: Traceback (most recent call last):
C:\Users\KingM\WPI\HU3910\IMAGS\gsr\collect_gsr.py", line 108, in <module>
()
C:\Users\KingM\WPI\HU3910\IMAGS\gsr\collect_gsr.py", line 103, in main
= pick_port_interactively()
~~~~~
C:\Users\KingM\WPI\HU3910\IMAGS\gsr\collect_gsr.py", line 28, in pick_port_interact
lce = input("Select port number: ").strip()
~~~~~
Interrupt
PS C:\Users\KingM\WPI\HU3910\IMAGS\gsr> python collect_gsr.py
⊗ 0 0 - VIM: DISABLED - 5<ExtensionDisable>
ASUS Zenbook
F1 F2 F3 F4 F5 F6 F7 F8
```

Live GSR plotting:

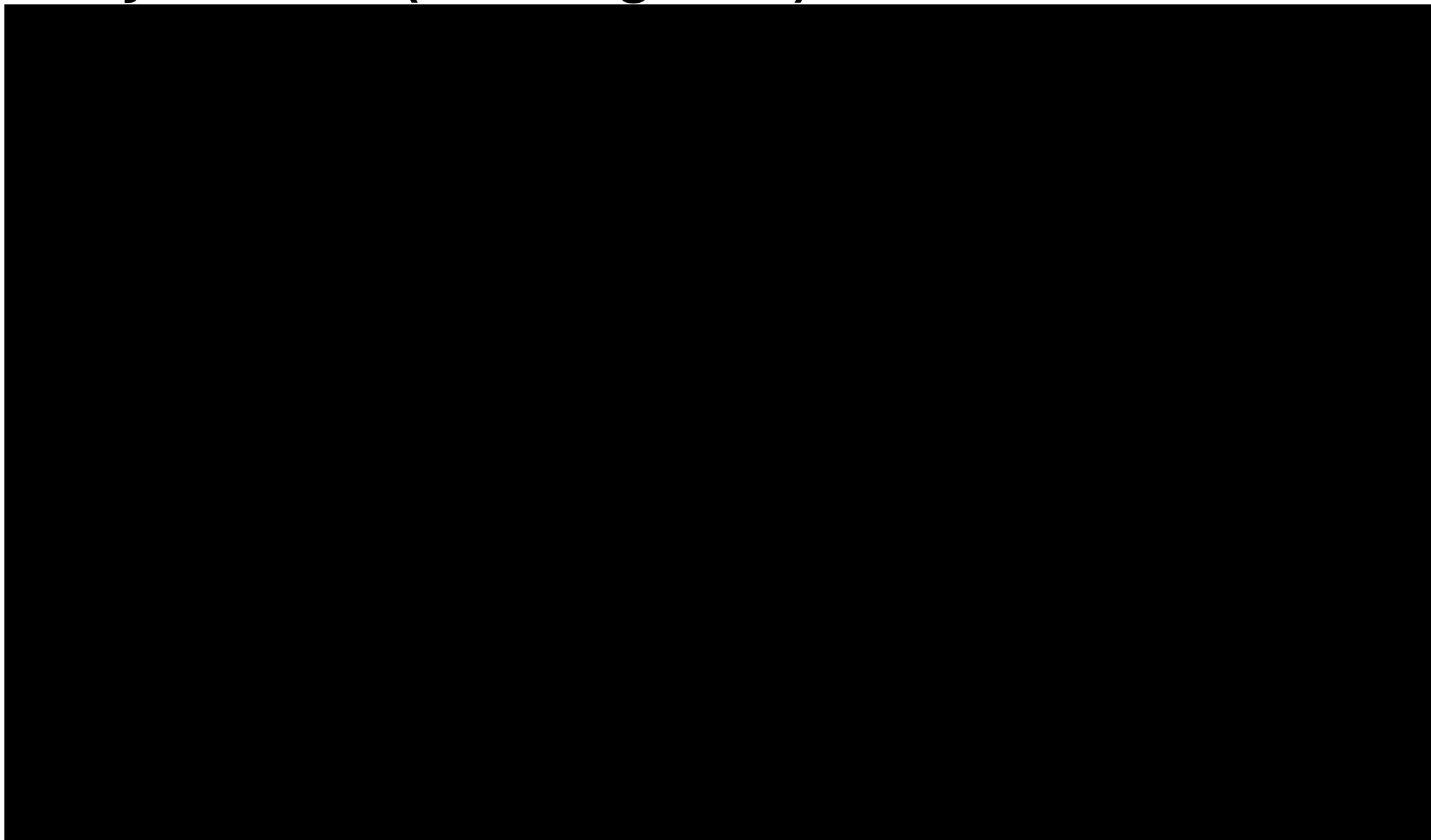
Smooth GSR



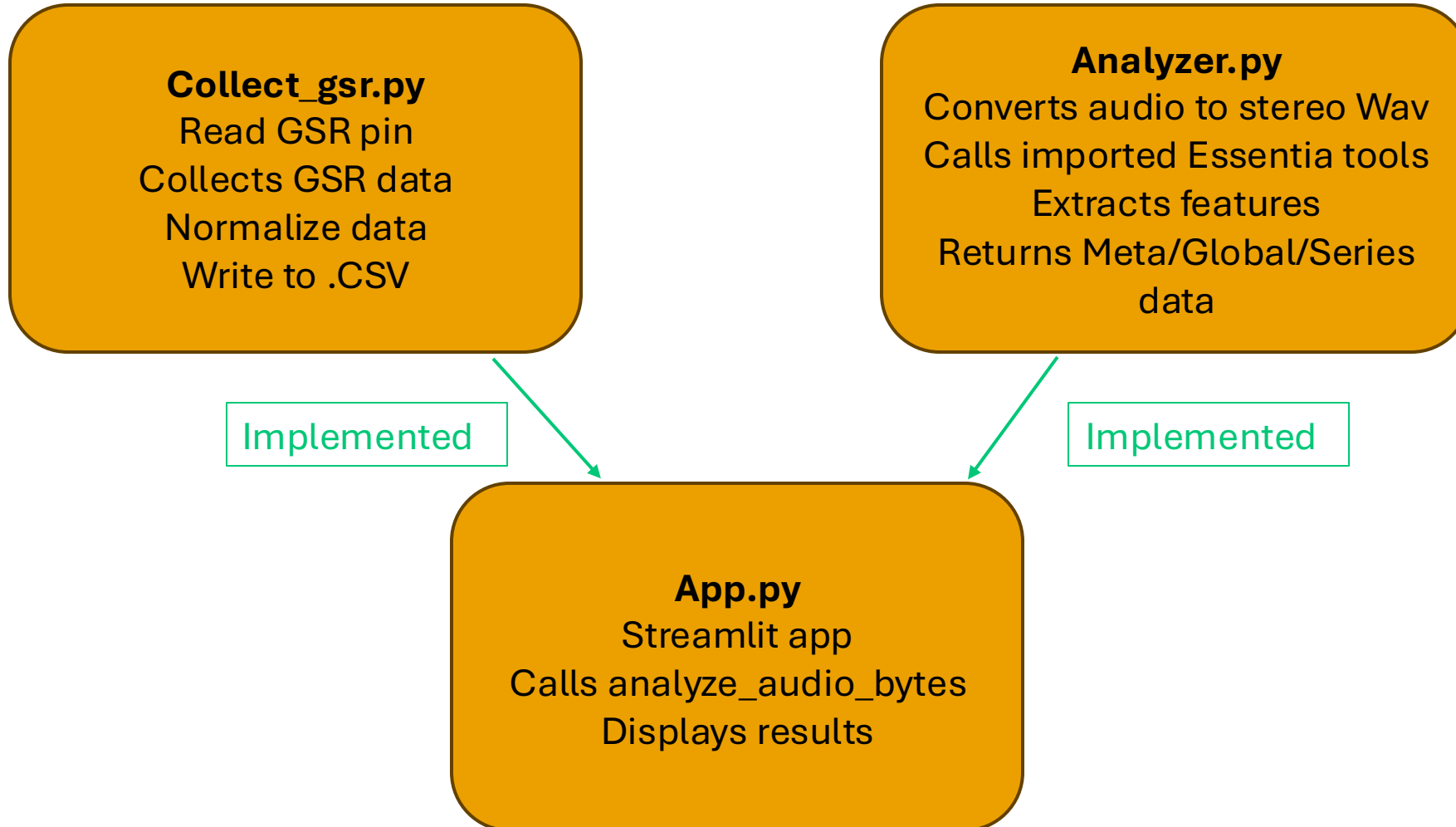
The screenshot shows a VS Code editor with a Python file named `collect_gsr.py`. The code defines a `parse_line` function to handle incoming data and a `run_live_plot` function to stream data to a plot. The `run_live_plot` function uses `matplotlib` to create a plot with 'Time (s)' on the x-axis and 'GSR (raw units)' on the y-axis. The terminal window shows the execution of the script, which prompts for a port number (COM3) and starts streaming data. The keyboard of an ASUS Zenbook is visible at the bottom of the image.

```
File Edit Selection View Go Run ...  
gsr_reader.py app.py collect_gsr.py U X  
gsr > collect_gsr.py > run_live_plot  
36 def parse_line(line: str) -> Optional[float]:  
44     except ValueError:  
45         return None  
46  
47  
48 def run_live_plot(port: str, cfg: Config):  
49     window_size = 50  
50     buf = deque(maxlen=window_size)  
51  
52     ser = serial.Serial(port, cfg.baud, timeout=1)  
53     time.sleep(2.0) # allow Arduino reset  
54     ser.reset_input_buffer()  
55  
56     print("Streaming... Close plot window or press Ctrl+C to stop.")  
57  
58     plt.ion()  
59     fig, ax = plt.subplots()  
60     x_data, y_data = [], []  
61     (line,) = ax.plot([], [], linewidth=1)  
62  
63     ax.set_xlabel("Time (s)")  
64     ax.set_ylabel("GSR (raw units)")  
65  
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS  
(.venv) PS C:\Users\King\WPI\H3910\IMAGS\gsr> python collect_gsr.py  
File "C:\Users\King\WPI\H3910\IMAGS\gsr\collect_gsr.py", line 28, in pick_port_inter  
choice = input("Select port number: ").strip()  
KeyboardInterrupt  
(.venv) PS C:\Users\King\WPI\H3910\IMAGS\gsr> python collect_gsr.py  
Available serial ports:  
[0] COM3 (USB Serial Device (COM3))  
Select port number: 0  
Streaming... Close plot window or press Ctrl+C to stop.  
Stopped.  
(.venv) PS C:\Users\King\WPI\H3910\IMAGS\gsr>  
Dylan VM: DISABLED  
ASUS Zenbook  
Esc F1 F2 F3 F4 F5 F6 F7  
1 2 3 4 5 6 7  
Q W E R T Y  
A S D F G
```

Song Analyzer + GSR (Pre-Integration)



Our new IMAGS Structure



Continuation of the Project

- Documented report:
 - Integrate MIR and Analysis to Existing IMAGS Framework
 - Add User Survey Questions
 - Expand Music Analysis
 - Explore More Sensors